

Q Write a C program to simulate:

deadlock detection.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 10
```

```
int main()
```

```
{  
    int n, m;
```

```
    int allocation[MAX][MAX], request[MAX][MAX];
```

```
    int available[MAX];
```

```
    int work[MAX];
```

```
    int finish[MAX];
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter Allocation Matrix: \n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {  
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &allocation[i][j]);
```

```
    }  
    printf("Enter request Matrix: \n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {  
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &request[i][j]);
```

```
    }
```

```
}
```

```
printf("Enter Available Resources: \n");
```

```
for (int i = 0; i < m; i++)
```

```
    scanf("%d", &available[i]);
```

```
}
```



```
for (int i=0; i<m; i++) {
```

```
    sum += C[i];
```

```
    work[i] = available[i];
```

```
}
```

```
for (int i=0; i<n; i++) {
```

```
    finish[i] = false;
```

```
}
```

```
for (int k=0; k<n; k++) {
```

```
    for (int i=0; i<n; i++) {
```

```
        if (!finish[i]) {
```

```
            bool canExecute = true;
```

```
            for (int j=0; j<m; j++) {
```

```
                if (request[i][j] > work[j]) {
```

```
                    canExecute = false;
```

```
                    break;
```

```
            }
```

```
            if (canExecute) {
```

```
                for (int j=0; j<m; j++) {
```

```
                    work[j] += allocation[i][j];
```

```
                }
```

```
                finish[i] = true;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
bool deadlock = false;
```

```
printf("In Deadlocked Processes : ");
```

```
for (int i=0; i<n; i++) {
```

```
    if (!finish[i]) {
```

```
        printf("P%d ", i);
```

```
        deadlock = true;
```

```
}
```

```
}
```



```
if (!deadlock) {
```

```
printf("None");
```

```
printf("In System & NOT in deadlock.\n");
```

```
} else {
```

```
printf("In System & in DEADLOCK.\n");
```

```
}
```

```
return 0;
```

```
}
```

Enter number of processes: 4

Enter number of resources: 3

Enter Allocation Matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter Request Matrix:

0 0 1

1 0 2

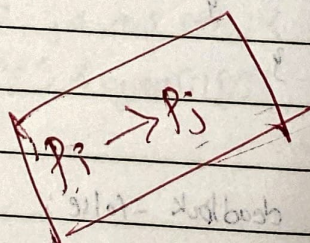
0 0 0

3 3 0

Enter available:

0 0 0

Process P3 is in deadlock.



$Req \leq Need$ if both satisfied then p provide request
 $Req \leq available$ if both satisfied then p provide request
 ,, available = available - request
 Allocation = allocation + request
 Need = Need - request

Process	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P ₀	1	0	2	0	0	1	0	0	0
P ₁	2	1	1	1	0	2			
P ₂	1	0	3	0	0	0			
P ₃	1	2	2	3	3	0			

P₀ available = 000

Request ≤ available

(0, 0, 1) ≤ (0, 0, 0)

finish[0] = false

P₁ available = 000

Req ≤ available

(1, 0, 2) ≤ (0, 0, 0)

finish[1] = false

P₂ = available = 0, 0, 0

Req ≤ ^{available} (0, 0, 0)

(0, 0, 0) ≤ (0, 0, 0) ✓

Work = work + allocation

= 000 + 103

= (1, 0, 3)

P₃ available = (1, 0, 3)

req ≤ available

(1, 2, 2) ≤ (1, 0, 3)

P₀ Req ≤ available

(0, 0, 1) ≤ (1, 0, 3)

work = 1, 0, 3 + 1, 0, 2 = 2, 0, 5

≤ 2, 0, 5

P₃ = Req ≤ available

3, 3, 0 ≤ (4, 1, 6) ✗

∴ P₃ is in deadlock.